

Cours sur les arbres en Python

Table des matières

1	Introduction et motivation (10-15 min)	2
1.1	Pourquoi les arbres ?	2
1.2	Comparaison avec les structures linéaires	2
1.3	Cas concrets d'utilisation	2
1.4	Terminologie de base	2
1.5	Différents types d'arbres	2
2	Concepts fondamentaux et implémentation (30-35 min)	3
2.1	Implémentation d'un arbre en Python	3
2.2	Parcours d'un arbre	4
2.2.1	Parcours en profondeur (DFS)	4
2.2.2	Parcours en largeur (BFS)	5
2.3	Arbre Binaire de Recherche (BST)	6
2.4	Suppression dans un BST	7
3	Comparaison avec d'autres structures (10 min)	9
3.1	Complexités algorithmiques	9
3.2	Exemples concrets	9
3.2.1	Tri d'éléments	9
3.2.2	Système de fichiers hiérarchique	10
4	Exercices pratiques	11
4.1	Exercice 1 : Hauteur d'un arbre	11
4.2	Exercice 2 : Vérification BST	11
5	Conclusion	12

1 Introduction et motivation (10-15 min)

1.1 Pourquoi les arbres ?

Les arbres sont des structures de données non linéaires qui organisent les données de manière hiérarchique. Contrairement aux listes (linéaires) ou aux dictionnaires (associatifs), les arbres permettent de modéliser des relations parent-enfant.

1.2 Comparaison avec les structures linéaires

Structure	Avantages	Inconvénients
Listes	Simple, accès séquentiel	Recherche lente ($O(n)$)
Dictionnaires	Accès rapide ($O(1)$)	Pas de relation hiérarchique
Arbres	Recherche hiérarchique, naturel	Plus complexe à implémenter

TABLE 1 – Comparaison des structures de données

1.3 Cas concrets d'utilisation

- Arbres généalogiques : Relations familiales
- Hiérarchie d'entreprise : Structure organisationnelle
- Systèmes de fichiers : Répertoires et sous-répertoires
- Moteurs de recherche : Indexation de pages web
- Bases de données : Index B-tree, B+tree

1.4 Terminologie de base

1.5 Différents types d'arbres

- Arbre binaire : Chaque nœud a au plus 2 enfants

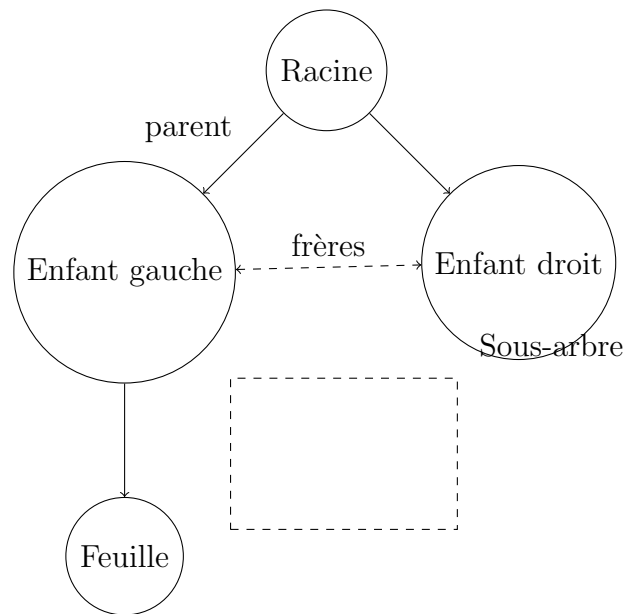


FIGURE 1 – Terminologie d'un arbre

- **Arbre binaire de recherche (BST)** : Nœuds ordonnés
- **Arbre équilibré (AVL)** : Hauteur contrôlée
- **Arbre rouge-noir** : Équilibrage approximatif
- **Arbre B** : Pour les systèmes de fichiers et bases de données

2 Concepts fondamentaux et implémentation (30-35 min)

2.1 Implémentation d'un arbre en Python

```

1 class Node:
2     """Classe représentant un nœud d'arbre binaire"""
3     def __init__(self, value):
4         self.value = value # Valeur du nœud
5         self.left = None # Enfant gauche
6         self.right = None # Enfant droit
7
8     def __str__(self):
9         return f"Node({self.value})"

```

```

10
11 # Exemple de cr ation d'un arbre
12 def create_example_tree():
13     """Cr e un arbre binaire simple"""
14     root = Node(10)
15     root.left = Node(5)
16     root.right = Node(15)
17     root.left.left = Node(3)
18     root.left.right = Node(7)
19     root.right.left = Node(12)
20     root.right.right = Node(18)
21     return root

```

Listing 1 – Structure de base d'un nœud

```

1 """
2
3      10
4     / \
5    5   15
6   / \ / \
7  3  7 12 18
8 """

```

Listing 2 – Représentation ASCII de l'arbre

2.2 Parcours d'un arbre

2.2.1 Parcours en profondeur (DFS)

```

1 def preorder(node):
2     """Parcours pr -ordre : Racine      Gauche      Droite"""
3     if node:
4         print(node.value, end=" ")
5         preorder(node.left)
6         preorder(node.right)
7
8 def inorder(node):
9     """Parcours en-ordre : Gauche      Racine      Droite"""
10    if node:
11        inorder(node.left)
12        print(node.value, end=" ")
13        inorder(node.right)

```

```

14
15 def postorder(node):
16     """Parcours post-ordre : Gauche      Droite      Racine"""
17     if node:
18         postorder(node.left)
19         postorder(node.right)
20         print(node.value, end=" ")
21
22 # Exemple d'utilisation
23 root = create_example_tree()
24 print("Pr -ordre:", end=" ")
25 preorder(root) # 10 5 3 7 15 12 18
26
27 print("\nEn-ordre:", end=" ")
28 inorder(root) # 3 5 7 10 12 15 18
29
30 print("\nPost-ordre:", end=" ")
31 postorder(root) # 3 7 5 12 18 15 10

```

Listing 3 – Parcours DFS

2.2.2 Parcours en largeur (BFS)

```

1 from collections import deque
2
3 def bfs(root):
4     """Parcours en largeur niveau par niveau"""
5     if not root:
6         return
7
8     queue = deque([root])
9
10    while queue:
11        node = queue.popleft()
12        print(node.value, end=" ")
13
14        if node.left:
15            queue.append(node.left)
16        if node.right:
17            queue.append(node.right)
18
19 # Exemple
20 root = create_example_tree()
21 print("BFS:", end=" ")

```

```
22 bfs(root)  # 10 5 15 3 7 12 18
```

Listing 4 – Parcours BFS

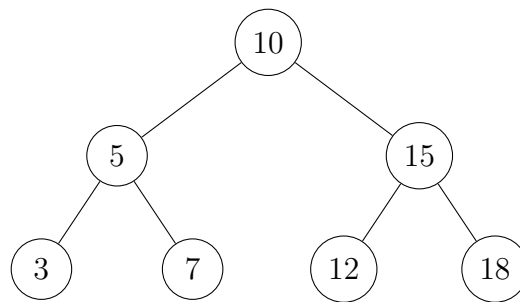


FIGURE 2 – Arbre binaire exemple

2.3 Arbre Binaire de Recherche (BST)

```
1 class BinarySearchTree:
2     """Arbre Binaire de Recherche"""
3     def __init__(self):
4         self.root = None
5
6     def insert(self, value):
7         """Ins re une valeur dans le BST"""
8         self.root = self._insert_recursive(self.root, value)
9
10    def _insert_recursive(self, node, value):
11        """M thode r cursive d'insertion"""
12        if not node:
13            return Node(value)
14
15        if value < node.value:
16            node.left = self._insert_recursive(node.left,
17            value)
18        elif value > node.value:
19            node.right = self._insert_recursive(node.right,
20            value)
21
22        return node
23
24    def search(self, value):
```

```

23     """Recherche une valeur dans le BST"""
24     return self._search_recursive(self.root, value)
25
26     def _search_recursive(self, node, value):
27         """Methode recursive de recherche"""
28         if not node or node.value == value:
29             return node
30
31         if value < node.value:
32             return self._search_recursive(node.left, value)
33         return self._search_recursive(node.right, value)
34
35     def find_min(self):
36         """Trouve la valeur minimale"""
37         if not self.root:
38             return None
39
40         current = self.root
41         while current.left:
42             current = current.left
43         return current.value
44
45 # Exemple d'utilisation
46 bst = BinarySearchTree()
47 values = [10, 5, 15, 3, 7, 12, 18]
48 for v in values:
49     bst.insert(v)
50
51 print("Recherche de 7:", bst.search(7) is not None) # True
52 print("Valeur minimale:", bst.find_min())           # 3

```

Listing 5 – Implémentation d'un BST

2.4 Suppression dans un BST

```

1 def delete_node(root, value):
2     """Supprime un nœud d'un BST"""
3     if not root:
4         return root
5
6     # Recherche du nœud à supprimer
7     if value < root.value:
8         root.left = delete_node(root.left, value)
9     elif value > root.value:

```

```

10     root.right = delete_node(root.right, value)
11 else:
12     # N ud trouv
13     # Cas 1: Pas d'enfant ou un seul enfant
14     if not root.left:
15         return root.right
16     elif not root.right:
17         return root.left
18
19     # Cas 2: Deux enfants
20     # Trouver le successeur (minimum du sous-arbre droit)
21     temp = find_min_node(root.right)
22     root.value = temp.value
23     root.right = delete_node(root.right, temp.value)
24
25     return root
26
27 def find_min_node(node):
28     """Trouve le n ud minimum"""
29     current = node
30     while current.left:
31         current = current.left
32     return current
33
34 # Exemple de suppression
35 root = create_example_tree()
36 print("Avant suppression - en ordre:", end=" ")
37 inorder(root) # 3 5 7 10 12 15 18
38
39 root = delete_node(root, 5)
40 print("\nApr s suppression de 5 - en ordre:", end=" ")
41 inorder(root) # 3 7 10 12 15 18

```

Listing 6 – Suppression dans un BST

Opération	Liste	Dictionnaire	BST (équilibré)
Recherche	$O(n)$	$O(1)$	$O(\log n)$
Insertion	$O(1)^*$	$O(1)$	$O(\log n)$
Suppression	$O(n)$	$O(1)$	$O(\log n)$
Tri	$O(n \log n)$	N/A	$O(n)$ (parcours)

TABLE 2 – Comparaison des complexités (* : $O(1)$ pour insertion en fin de liste)

3 Comparaison avec d'autres structures (10 min)

3.1 Complexités algorithmiques

3.2 Exemples concrets

3.2.1 Tri d'éléments

```

1 import time
2 import random
3
4 def sort_with_bst(numbers):
5     """Tri utilisant un BST"""
6     bst = BinarySearchTree()
7     for num in numbers:
8         bst.insert(num)
9
10    sorted_list = []
11    def inorder_collect(node):
12        if node:
13            inorder_collect(node.left)
14            sorted_list.append(node.value)
15            inorder_collect(node.right)
16
17    inorder_collect(bst.root)
18    return sorted_list
19
20 # Comparaison de performance
21 numbers = random.sample(range(10000), 1000)
22
23 # Tri avec BST
24 start = time.time()

```

```

25 sorted_bst = sort_with_bst(numbers)
26 bst_time = time.time() - start
27
28 # Tri avec sorted()
29 start = time.time()
30 sorted_list = sorted(numbers)
31 list_time = time.time() - start
32
33 print(f"BST sort time: {bst_time:.4f}s")
34 print(f"List sort time: {list_time:.4f}s")
35 print(f"Les deux méthodes produisent le même résultat: {
    sorted_bst == sorted_list}")

```

Listing 7 – Tri avec BST vs liste

3.2.2 Système de fichiers hiérarchique

```

1 class FileSystemNode:
2     """Nœud pour système de fichiers"""
3     def __init__(self, name, is_file=False):
4         self.name = name
5         self.is_file = is_file
6         self.children = [] # Liste d'enfants
7         self.parent = None
8
9     def add_child(self, child):
10        """Ajoute un enfant au nœud"""
11        child.parent = self
12        self.children.append(child)
13
14    def display(self, level=0):
15        """Affiche l'arborescence"""
16        indent = "  " * level
17        prefix = "[F] " if self.is_file else "[D] "
18        print(f"{indent}{prefix}{self.name}")
19
20        for child in self.children:
21            child.display(level + 1)
22
23 # Création d'une structure de fichiers
24 root = FileSystemNode("/")
25 home = FileSystemNode("home")
26 user = FileSystemNode("user")
27 docs = FileSystemNode("Documents")

```

```

28 file1 = FileSystemNode("rapport.txt", is_file=True)
29 file2 = FileSystemNode("notes.txt", is_file=True)
30
31 root.add_child(home)
32 home.add_child(user)
33 user.add_child(docs)
34 docs.add_child(file1)
35 docs.add_child(file2)
36
37 print("Structure du syst me de fichiers:")
38 root.display()

```

Listing 8 – Modélisation d'un système de fichiers

4 Exercices pratiques

4.1 Exercice 1 : Hauteur d'un arbre

```

1 def tree_height(node):
2     """Calcule la hauteur d'un arbre"""
3     if not node:
4         return 0
5     return 1 + max(tree_height(node.left), tree_height(node.
6                     right))
7
8 # Test
9 root = create_example_tree()
10 print(f"Hauteur de l'arbre: {tree_height(root)}") # 3

```

Listing 9 – Calcul de la hauteur

4.2 Exercice 2 : Vérification BST

```

1 def is_bst(node, min_val=float('-inf'), max_val=float('inf'))
2 :
3     """V rifie si un arbre est un BST valide"""
4     if not node:
5         return True
6
7     if node.value <= min_val or node.value >= max_val:
8         return False

```

```

8
9     return (is_bst(node.left, min_val, node.value) and
10             is_bst(node.right, node.value, max_val))
11
12 # Test avec différents arbres
13 valid_bst = create_example_tree()
14 print(f"Arbre valide BST: {is_bst(valid_bst)}") # True
15
16 # Création d'un arbre non-BST
17 invalid_root = Node(10)
18 invalid_root.left = Node(15) # Violation: 15 > 10 dans sous-
    arbre gauche
19 invalid_root.right = Node(20)
20 print(f"Arbre invalide BST: {is_bst(invalid_root)}") # False

```

Listing 10 – Vérification si un arbre est un BST

5 Conclusion

Les arbres sont des structures fondamentales en informatique avec de nombreuses applications pratiques. Leur compréhension est essentielle pour :

- Optimiser les recherches dans de grandes collections de données
- Modéliser des relations hiérarchiques naturelles
- Implémenter des algorithmes efficaces de tri et de recherche
- Comprendre des structures plus avancées (grappes, réseaux neuronaux, etc.)

Le BST, bien que simple, est la base d'arbres plus sophistiqués comme les AVL et les arbres rouge-noir utilisés dans les bibliothèques standard de nombreux langages.

6 Références

- Cormen, T. H., et al. "Introduction to Algorithms"
- Knuth, D. E. "The Art of Computer Programming" é *Python Documentation : collections deque*
- *GeeksforGeeks : Binary Tree Data Structure*